

Building Professional CLIs in Python

Avik Basu

Staff Data Scientist

Intuit



About Me

- Based in Sunnyvale, CA
- Staff Data Scientist at Intuit
- Editor at PyOpenSci
- Love RPG!
- Driving is therapy! 📍



Agenda

1. **Why CLIs?** - Motivation
2. **Argparse** – Baseline functionality
3. **Typer** – Function signatures as CLIs
4. **Rich** – Output that communicates
5. **Textual** – Interactive TUIs
6. **Production patterns** – How to scale things up!



My first CLI experience

- Year **2013**
- Just installed **Ubuntu**
- First **Python project**
- Learning **Git** as a CLI



My first CLI experience

- Year **2013**
- Just Installed **Ubuntu**
- First **Python project**
- Learning **Git** as a CLI

```
→ cool-project git:(master) ✕ git status  
On branch master
```

```
No commits yet
```

```
Changes to be committed:
```

```
(use "git rm --cached <file>..." to unstage)
```

```
new file:   requirements.txt
```

```
new file:   src/data.py
```

```
new file:   src/model.py
```

```
→ cool-project git:(master) ✕ git commit
```

I typed `git`
`commit...`

...and suddenly I was
here



I typed `git`
`commit`...

...and suddenly I was
here

```
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch master
#
# Initial commit
#
# Changes to be committed:
#       new file:   requirements.txt
#       new file:   src/data.py
#       new file:   src/model.py
#
~
~
~
~
~
~
```

OK, I wrote a
message...

...now **how do I save
and exit?**

```
first commit wohoooo!  
# Please enter the commit message for your changes. Lines starting  
# with '#' will be ignored, and an empty message aborts the commit.  
#  
# On branch master  
#  
# Initial commit  
#  
# Changes to be committed:  
#       new file:   requirements.txt  
#       new file:   src/data.py  
#       new file:   src/model.py  
#  
~  
~  
~  
~  
~  
~  
~
```


Why great CLIs matter

- Fast
- Composable
- Scriptable
- Universal
- Agent-contract



Our Running Example

A tool to monitor ML model explainability

- Capture explanations as the model runs in production
- Understand which features drive the predictions
- Compare explanations across different model versions or time windows



Three Commands

Command	What it does
<code>log</code>	Log explanation values for a batch of predictions
<code>report</code>	Generate summary and drift reports (2 sub-commands)
<code>watch</code>	Live monitoring dashboard

No ML background needed. Just think of it as:

log data → analyze data → watch data live



What we want from our CLI

1. **Distributable** in a venv and globally
2. **Support Subcommands**
3. **Type-safe inputs**
4. **Config hierarchy**
5. **Progress feedback**
6. **Fast**
7. **Good defaults**



Make it a command

```
1 [project.scripts]
2 shapmonitor = "shapmonitor.cli:app"
```

- `pip install` puts `shapmonitor` on `$PATH`.
- For local hacking: `python -m shapmonitor` works too if there is a `__main__.py` setup



Argparse

Imperative Style



The log command, with argparse

```
1 # shapmonitor/__main__.py
2 import argparse, os, sys
3
4 def main():
5     parser = argparse.ArgumentParser(
6         prog="shapmonitor",
7         description="Log SHAP values for a batch of predictions.",
8     )
9     parser.add_argument("data", help="Path to CSV file with feature data")
10    parser.add_argument("--model", required=True, help="Path to model file")
11    parser.add_argument("--data-dir", default="./shap_logs")
12    # more args...
13
14    args = parser.parse_args()
15
16    if not os.path.exists(args.data):
17        print(f"Error: data file not found: {args.data}", file=sys.stderr)
18        sys.exit(1)
19    # same for model
20    if not 0.0 ≤ args.sample_rate ≤ 1.0:
21        print("Error: --sample-rate must be between 0.0 and 1.0", file=sys.stderr)
22        sys.exit(1)
23    # ... and *now* we can read the CSV and compute SHAP values
```

~20 lines of plumbing before we touch the data.

Help command - built-in

```
1 $ shapmonitor log --help
```

```
1 usage: shapmonitor log [-h] --model MODEL
2 [--data-dir DATA_DIR] [--sample-rate SAMPLE_RATE]
3 [--model-version MODEL_VERSION] data
4
5 positional arguments:
6   data                  Path to CSV file with feature data
7
8 options:
9   -h, --help            show this help message and exit
10  --model MODEL          Path to pickled model
11  --data-dir DATA_DIR  Output directory
12  --sample-rate SAMPLE_RATE
13                        Sampling rate (0.0-1.0)
14  --model-version MODEL_VERSION
15                        Model version tag
```


The **log** command

```
1 | $ shapmonitor log demo/batch_current.csv --model demo/model.pkl --data-dir ./logs
```

```
1 | Logged 100 samples to ./logs
```

It writes the explanation (SHAP) values to disk.



The **report summary** command

```
1 $ shapmonitor report summary --data-dir ./logs
```

```
1 Feature                Mean |SHAP|      Mean      Std
2 -----
3 age                    0.1159    0.0106    0.1298
4 recent_inquiries      0.1053   -0.0134    0.1165
5 employment_years      0.0806    0.0136    0.1001
6 credit_score          0.0563   -0.0075    0.0673
7 payment_history       0.0476   -0.0178    0.0612
8 debt_ratio            0.0340   -0.0187    0.0427
9 income                0.0120   -0.0003    0.0183
10 num_accounts         0.0054   -0.0001    0.0088
```

It works but, **we can do better.**

The pain points

1. **Verbose argument definitions**
2. **Manual validation**
3. **No Config hierarchy**
4. **Text-only output**

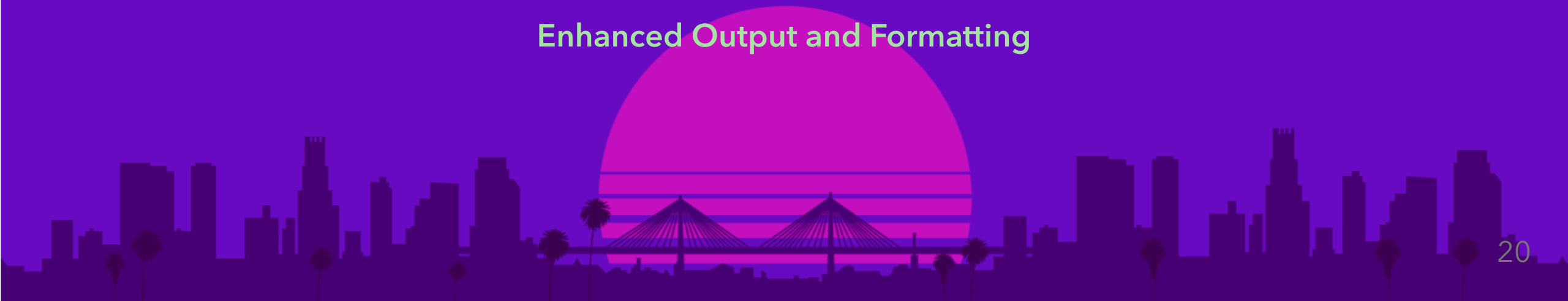


Typer

Core Idea: The Function Signature is the CLI

Rich

Enhanced Output and Formatting



```
1  # shapmonitor/cli/log.py
2  from pathlib import Path
3  from typing import Annotated
4  import typer
5
6  def log_command(
7      data: Annotated[Path, typer.Argument(exists=True, readable=True)],
8      model: Annotated[Path, typer.Option("--model", "-m",
9          exists=True, readable=True)],
10     data_dir: Annotated[Path, typer.Option(
11         envvar="SHAPMONITOR_DATA_DIR")] = Path("./shap_logs"),
12     sample_rate: Annotated[float, typer.Option(min=0.0, max=1.0)] = 1.0,
13 ) → None:
14     """Log SHAP values for a batch of predictions."""
15     ...
```

Type hints become **CLI arguments**. Validation is **Declarative**.

Subcommands and composition

```
1  # shapmonitor/cli/report.py - a nested Typer group
2  report_app = typer.Typer(no_args_is_help=True)
3
4  @report_app.command("summary")
5  def summary_command(...): ...
6
7  @report_app.command("drift")
8  def drift_command(...): ...
9
10 # shapmonitor/cli/__init__.py - mount the group under "report"
11 app.add_typer(report_app, name="report")
```

```
1  → shapmonitor report summary --top-k 5
2  → shapmonitor report drift --ref last-14d..last-7d
```

Some Good Typer Defaults

```
1  # shapmonitor/cli/__init__.py
2  app = typer.Typer(
3      name="shapmonitor",
4      no_args_is_help=True,      # no args? show help, don't error
5      help="Monitor SHAP explanations over time.",
6      rich_markup_mode="rich",  # styled help output
7  )
```

- `no_args_is_help=True` – bare `shapmonitor` prints help, not a usage error
- `rich_markup_mode="rich"` – colored, boxed help output from the docstring

Enhanced Output

Rich



Makes life better in a terminal

Rich provides:

- **Progress bars** – feedback for long-running tasks
- **Tables** – structured data that's easy to scan
- **Panels** – grouped information with borders
- **Styled text** – emphasis, color-coded severity
- **Tracebacks** – readable, syntax-highlighted errors



Status Indicators

```
1 # shapmonitor/cli/log.py
2 from rich.progress import Progress, SpinnerColumn, TextColumn
3
4 with Progress(
5     SpinnerColumn(),
6     TextColumn("[progress.description]{task.description}"),
7     console=console,
8     transient=True, # disappears when done
9 ) as progress:
10     progress.add_task("Computing SHAP values...", total=None)
11     monitor.log_batch(X)
```

```
1  ⋮ Computing SHAP values...
```

- **Transient spinners** for indeterminate work.
- **Progress bars** when you know the total.

Tables with visual weight

```
1  # shapmonitor/cli/report.py - summary_command
2  from rich.table import Table
3
4  table = Table(title=f"SHAP Summary ({p.start} → {p.end})")
5  table.add_column("Feature", style="bold")
6  table.add_column("Mean |SHAP|", justify="right")
7  table.add_column("", justify="left")          # inline bar
8  table.add_column("Mean", justify="right")
9  table.add_column("Std", justify="right")
10
11  for feat, row in summary_df.iterrows():
12      bar_len = int(20 * row["mean_abs"] / max_mean_abs)
13      bar = "[cyan]" + "\u2588" * bar_len + "[/cyan]"
14      table.add_row(str(feat), f"{row['mean_abs']:.4f}", bar,
15                    f"{row['mean']:.4f}", f"{row['std']:.4f}")
16
17  console.print(table)
```

[(shap-monitor-py3.13) → shap-monitor git:(feat/cli) ✕ shapmonitor report summary]

Samples: 5200

SHAP Summary (2026-04-11 00:00:00 → 2026-04-18 00:00:00)

Feature	Mean SHAP		Mean	Std
recent_inquiries	0.1110		0.0287	0.1180
age	0.1057		0.0015	0.1191
employment_years	0.0689		0.0050	0.0880
payment_history	0.0604		-0.0402	0.0687
debt_ratio	0.0559		-0.0419	0.0605
credit_score	0.0495		0.0014	0.0607
income	0.0077		-0.0012	0.0118
num_accounts	0.0050		-0.0017	0.0076

Styled status and panels

```
1  # shapmonitor/cli/report.py - drift_command
2  from rich.panel import Panel
3
4  alerts = int((drift_df["psi"] ≥ 0.25).sum())
5  warnings = int(((drift_df["psi"] ≥ 0.1) & (drift_df["psi"] < 0.25)).sum())
6  stable = int((drift_df["psi"] < 0.1).sum())
7
8  headline = " . ".join([
9      f"[red]{alerts} alert{'s' if alerts ≠ 1 else ''}[/red]",
10     f"[yellow]{warnings} warning{'s' if warnings ≠ 1 else ''}[/yellow]",
11     f"[green]{stable} stable[/green]",
12 ])
13
14 console.print(Panel(headline, title="Drift Report", border_style="blue"))
```

```
(shap-monitor-py3.13) → shap-monitor git:(feat/cli) ✖ shapmonitor report drift \
[--data-dir demo/shap_logs --ref last-14d..last-7d --curr last-7d..now]
```

Drift Report (ref: 2026-04-04 00:00:00→2026-04-11 00:00:00 vs curr: 2026-04-11 00:00:00)
0 alerts · 2 warnings · 6 stable

Feature	PSI	Status	SHAP ref	SHAP curr	Δ SHAP	Rank Δ
payment_history	0.2356	warning	0.0406	0.0539	0.0134	—
credit_score	0.1010	warning	0.0580	0.0553	-0.0027	—
employment_years	0.0832	stable	0.0791	0.0910	0.0118	—
num_accounts	0.0498	stable	0.0059	0.0043	-0.0016	—
debt_ratio	0.0414	stable	0.0343	0.0336	-0.0006	—
recent_inquiries	0.0294	stable	0.1041	0.0990	-0.0051	—
age	0.0205	stable	0.1154	0.1178	0.0024	—
income	0.0085	stable	0.0127	0.0130	0.0003	—

Interactive TUI

Textual



When static output isn't enough

Some tasks need **interactivity**:

- **Live monitoring** – data that updates in real time
- **Exploration** – filter, sort, navigate large datasets
- **Keyboard-driven workflows** – no mouse, no re-running

Textual gives you a full application framework in the terminal: widgets, layouts, events, CSS styling, keybindings.



App structure

```
1  # shapmonitor/cli/_watch_app.py
2  from textual.app import App, ComposeResult
3  from textual.widgets import DataTable, Footer, Header, Input
4
5  class WatchApp(App):
6      TITLE = "shapmonitor watch"
7      CSS = """
8      #filter-input { dock: top; margin: 0 1; height: 3; }
9      DataTable { height: 1fr; }
10     """
11     def compose(self) → ComposeResult:
12         yield Header(show_clock=True)
13         yield Input(placeholder="Type to filter features...", id="filter-input")
14         yield DataTable(id="shap-table")
15         yield Footer()
```

- `compose()` defines the layout.
- CSS controls sizing.

Live data with auto-refresh

```
1  def on_mount(self) → None:
2      table = self.query_one(DataTable)
3      table.add_columns("Feature", "Mean |SHAP|", "Std", "PSI", "Status")
4
5      self._load_data() # initial load
6      self.set_interval(self._refresh_interval, self._load_data)
7
8  def _load_data(self) → None:
9      analyzer = SHAPAnalyzer(get_backend(self._data_dir))
10     period = parse_period(self._period_spec)
11     self._summary = analyzer.summary(period.start, period.end)
12     self._refresh_table()
```

The table rebuilds with fresh data every 5 seconds.

Production Patterns



Composability – pipeable output

A great CLI plays well with `jq`, `xargs`, and humans at the same time.

```
1 if json_mode:
2     typer.echo(json.dumps(data, default=str))
3     return
4 render_rich_table(data)
```

```
1 $ shapmonitor report summary --json | jq '.features[:1]'
```

```
1 [
2   {
3     "feature": "recent_inquiries",
4     "mean_abs": 0.11099565029144287,
5     "mean": 0.02865222841501236,
6     "std": 0.11797577887773514,
7     "min": -0.2615908980369568,
8     "max": 0.23528572916984558
9   },
10 ]
```

Imports are EXPENSIVE for a CLI application

```
1  # shapmonitor/__init__.py
2  import logging
3  from shapmonitor.monitor import SHAPMonitor
4
5  logging.getLogger(__name__).addHandler(logging.NullHandler())
6
7  __all__ = ["SHAPMonitor"]
8
9  # 1. `import shapmonitor`
10 # 2. `from shapmonitor import shapmonitor`
11 # Both of them basically take the same time.
```

The cost: every **import** of the package pays for a lot of dependencies; even when the caller never touches the main module.

For a CLI, that means even **shapmonitor --help** can become slow.

Lazy Import Strategies

- Use local imports, e.g. inside a function
- Use the `lazy-loader` library
- Use `TYPE_CHECKING` for expensive type hints
- Wait for Python 3.15 (PEP 810)



Lazy module attributes (Thanks to: PEP 562 🙏)

```
1  # shapmonitor/__init__.py
2  __all__ = ["SHAPMonitor"]
3
4  def __getattr__(name):
5      if name == "SHAPMonitor":
6          from shapmonitor.monitor import SHAPMonitor
7          return SHAPMonitor
8      raise AttributeError(f"module {__name__!r} has no attribute {name!r}")
```

- `import shapmonitor` → loads only the module shell
- `from shapmonitor import SHAPMonitor` → triggers the dependency loading **only on first access**

Distribution

Not everyone is a python developer or a python user! 😊

```
1 | pip install shap-monitor          # library users
```

```
1 | pipx install "shap-monitor[cli]"  # CLI users installed globally
```

```
1 | uvx install "shap-monitor[cli]"  # CLI users installed globally
```

```
1 | brew install shap-monitor        # if needed
```


Config hierarchy

Flags → env vars → defaults - the precedence users expect.

```
1 data_dir: Annotated[
2     Path,
3     typer.Option(
4         envvar="SHAPMONITOR_DATA_DIR",    # ← falls back to env var
5     ),
6 ] = Path("./shap_logs")                  # ← falls back to default
```

```
1 # All three work, in priority order:
2 $ shapmonitor log data.csv --data-dir /custom/path
3 $ SHAPMONITOR_DATA_DIR=/custom/path shapmonitor log data.csv
4 $ shapmonitor log data.csv # uses ./shap_logs
```

One line gives us 3 config layers.

Where should your CLI's files live?

A proper CLI can have **three categories** of files:

- **Config** – settings the user edits
- **Data** – persistent state your tool writes
- **Cache** – regeneratable files

The **XDG Base Directory Spec** – Linux's standard home for each:

Kind	Env var	Default
config	<code>\$XDG_CONFIG_HOME</code>	<code>~/.config</code>
data	<code>\$XDG_DATA_HOME</code>	<code>~/.local/share</code>
cache	<code>\$XDG_CACHE_HOME</code>	<code>~/.cache</code>

Spec: specifications.freedesktop.org/basedir/latest



Cross-platform with `platformdirs`

```
1 from platformdirs import user_config_dir, user_data_dir, user_cache_dir
2
3 config_dir = Path(user_config_dir("your_app"))
4 data_dir   = Path(user_data_dir("your_app"))
5 cache_dir  = Path(user_cache_dir("your_app"))
6
7 # Linux:   ~/.config/yourapp, ~/.local/share/yourapp, ~/.cache/yourapp
8 # macOS:   ~/Library/Preferences/yourapp, ~/Library/Application Support/yourapp, ...
9 # Windows: %APPDATA%\yourapp, %LOCALAPPDATA%\yourapp, ...
```

Notable CLIs using this: `black`, `poetry`, `tox`.

When to reach for what

You need...	Use...	Complexity
Plain single script	Argparse	Low
Just argument parsing	Typer	Low
Readable, structured output	+ Rich	Low
Progress bars, spinners	+ Rich	Low
Live data, interactivity	+ Textual	Medium
Full TUI application	+ Textual	Higher



References

Typer – CLIs from type hints

Docs: typer.tiangolo.com · Source: github.com/tiangolo/typer

Rich – Beautiful terminal output

Docs: rich.readthedocs.io · Source: github.com/Textualize/rich

Textual – Full TUI application framework

Docs: textual.textualize.io · Source: github.com/Textualize/textual





Thank you! 🙏

Questions?



LinkedIn



Slides



shap-monitor

